



Sílabo de Curso

1ACB0002 - Arquitectura de Aplicaciones

I. INFORMACIÓN GENERAL

Nombre del Curso	:	Arquitectura de Aplicaciones
Código del curso	:	1ACB0002
Periodo	:	UG-1er Semestre 2026 Pregrado
Cuerpo académico	:	Garate Polar, Diego Alonso
Créditos	:	4
Semanas	:	16
Área o programa	:	INGENIERÍA DE CIBERSEGURIDAD
NRC	:	16505

II. MISIÓN Y VISIÓN DE LA UPC

Misión

Formar líderes íntegros e innovadores con visión global para que transformen el Perú.

Visión

Ser líder en la educación superior por su excelencia académica y su capacidad de innovación.

III. INTRODUCCIÓN

Descripción

El curso Arquitectura de Aplicaciones que toma la descripción de Desarrollo de Software Seguro introduce al estudiante en los principios, prácticas y herramientas necesarias para diseñar, desarrollar, desplegar y mantener software resiliente frente a amenazas de seguridad, incorporando la seguridad como un atributo transversal del ciclo de vida del software (Secure SDLC).

El curso enfatiza prevención, diseño seguro, codificación defensiva, modelado de amenazas, seguridad en APIs, y automatización DevSecOps, alineándose con estándares internacionales como OWASP, NIST y el Secure Software Development Framework (SSDF). Se combinan sesiones teóricas, análisis de casos reales y laboratorios prácticos orientados a entornos reales de desarrollo.

El curso se alinea explícitamente con el Student Outcome 5 de ABET, referido al trabajo efectivamente como miembro o líder en un equipo, cuyos miembros juntos proporcionan liderazgo, crea un entorno de colaboración e inclusión, establece metas, planifica tareas y cumple objetivos.. En este marco, los estudiantes desarrollarán la habilidad de analizar riesgos de seguridad en el desarrollo de software, diseñar controles adecuados y proponer soluciones seguras y viables, integrando principios de ingeniería de software y ciberseguridad.



Propósito

El propósito del curso es capacitar al estudiante para diseñar, desarrollar y evaluar software incorporando la seguridad como un requisito transversal a lo largo del ciclo de vida del desarrollo, con el fin de prevenir vulnerabilidades y reducir el riesgo de incidentes de ciberseguridad en entornos organizacionales. Mediante la aplicación de principios de diseño seguro, codificación segura, modelado de amenazas y prácticas de DevSecOps, el curso promueve un pensamiento crítico orientado a la prevención, resiliencia y mejora continua de la seguridad de las aplicaciones.

Asimismo, el estudiante desarrollará competencias clave de comunicación efectiva y resolución de problemas, permitiéndole analizar riesgos de seguridad, documentar hallazgos técnicos y comunicar recomendaciones claras y accionables a audiencias técnicas y no técnicas. Al finalizar el curso, estará preparado para proponer soluciones de desarrollo seguro que aporten valor tangible a los equipos de desarrollo y ciberseguridad, contribuyendo a la construcción de sistemas confiables, seguros y alineados con estándares internacionales.

Indicador

Curso de control de la competencia específica ABET 5 - Trabajo Multidisciplinario Nivel 1.

IV. LOGRO (S) DEL CURSO

Al finalizar el curso, el estudiante diseña y desarrolla aplicaciones aplicando principios de seguridad en el desarrollo desde el diseño hasta la puesta en producción, identificando y mitigando vulnerabilidades comunes, y comunicando riesgos y controles de manera efectiva.

V. COMPETENCIAS (S) DEL CURSO

Pensamiento Innovador - Nivel 2 - <p>Elabora una propuesta con características distintivas y multidisciplinaria, aplicando metodologías innovadoras para lograr un impacto positivo y argumentado en un contexto específico.
</p>

ABET 5 - Trabajo Multidisciplinario - - Nivel 1 - <p>Identifica los diferentes roles en el equipo, participa con dificultad en las actividades del grupo, de manera individual y con profesionales de otras áreas. Está dispuesto a escuchar las opiniones de sus compañeros e identifica los objetivos, actividades y plazos de entrega del trabajo desarrollado por los miembros del grupo.
</p>



VI. UNIDADES DE APRENDIZAJE

Unidad 1: Principios fundamentales del desarrollo de software seguro

COMPETENCIA (S):

Pensamiento Innovador

LOGRO DE LA UNIDAD: Al finalizar la unidad, el estudiante comprenderá y aplicará los principios fundamentales de seguridad en el diseño e implementación de software, analizando cómo estos influyen en la reducción de riesgos y vulnerabilidades desde etapas tempranas del desarrollo.

SEMANA	TEMARIO	ACTIVIDADES DE APRENDIZAJE	EVIDENCIAS DE APRENDIZAJE	BIBLIOGRAFÍA
1 - 3	- Introducción al desarrollo de software seguro: Relación entre ingeniería de software y ciberseguridad, Seguridad como requisito no funcional, Panorama de amenazas en aplicaciones modernas. - Principios de restricción en seguridad:- Least privilege
- Fail-safe defaults
- Complete mediation
- Separation
- Minimize trust - Simplicidad y metodología de diseño seguro:-	SESIÓN 1: Teoría - Introducción al desarrollo de software seguro - Rol del desarrollador en la postura de seguridad organizacional - Análisis de incidentes reales por fallas de desarrollo Laboratorio - Análisis guiado de incidentes reales - Identificación de errores de diseño vs implementación SESIÓN 2: Teoría - Principios de restricción y reducción de superficie de ataque		Bibliografía básica: NIST. <i>SP 800-218: Secure Software Development Framework (SSDF)</i>. OWASP. <i>OWASP Top 10 Web Application Security Risks</i>. McGraw, G. <i>Software Security: Building Security In</i>. Kohnfelder, L. <i>Designing Secure Software</i>. Bibliografía complementaria: Beyer et al. <i>Building Secure and Reliable Systems</i>. OWASP. <i>Attack Surface</i>



	Economy of mechanism - Minimize common mechanism - Least astonishment - Open design	Laboratorio - Análisis de un sistema simple identificando violaciones a principios de seguridad. Propuestas de solución. SESIÓN 3: Teoría - Relación entre complejidad y vulnerabilidades Laboratorio		Analysis Cheat Sheet. OWASP. Proactive Controls.
--	--	---	--	---

Unidad 2: Diseño de software seguro

COMPETENCIA (S):

Pensamiento Innovador

LOGRO DE LA UNIDAD: Al finalizar la unidad, el estudiante derivará y especificará requisitos de seguridad, y diseñará arquitecturas de software que incorporen controles de seguridad alineados a los objetivos del negocio y al contexto de uso del sistema.

SEMANA	TEMARIO	ACTIVIDADES DE APRENDIZAJE	EVIDENCIAS DE APRENDIZAJE	BIBLIOGRAFÍA
4 - 8	Derivación de requisitos de seguridad:- Relación entre riesgo del negocio y nivel de requisitos
- Requisitos de seguridad	Sesión 4: Teoría - Derivación de requisitos de seguridad - Activos, amenazas y riesgos - Seguridad derivada del negocio	- Evaluación Parcial 1 - Práctica Calificada 1 - Trabajo 1	Bibliografía básica: OWASP. Application Security Verification Standard (ASVS). NIST. SP 800-218: Secure Software Development



	dependen del contexto y del riesgo - Arquitectura de software segura:- Layering
- Abstraction
- Modularity
- Complete linkage - Lenguajes de programación y type safety:- Riesgos de los lenguajes
- Importancia del tipado seguro	- Especificación de requisitos de seguridad - Requisitos verificables - Seguridad como input para testing Laboratorio - Identificación de activos y requisitos de seguridad - Redacción de requisitos de seguridad - Revisión cruzada Sesión 5: Teoría Practica Calificada Laboratorio - Redacción de requisitos de seguridad - Revisión cruzada Sesión 6: Teoría - Arquitectura en capas y separación de responsabilidades	Framework (SSDF). Shostack, A. Threat Modeling: Designing for Security. Bass, L.; Clements, P.; Kazman, R. <i>Software Architecture in Practice</i>. Kohnfelder, L. <i>Designing Secure Software</i>. OWASP. <i>Secure Coding Practices</i>. Bibliografía complementaria: ISO/IEC 27005. Information Security Risk Management. SAFECode. Fundamental Practices for Secure Software Development.
--	--	--	---



		Laboratorio Revisión de arquitectura existente. Propuesta de mejoras Sesión 7: Teoría - Riesgos introducidos por los lenguajes - Importancia del tipado seguro - Errores comunes por falta de <i>type safety</i> Laboratorio Presentación Parcial del Proyecto Sesión 8:		
--	--	---	--	--

Unidad 3: Implementación Segura

COMPETENCIA (S):

Pensamiento Innovador

LOGRO DE LA UNIDAD: Al finalizar la unidad, el estudiante podrá implementar software aplicando prácticas de codificación segura, programación defensiva y uso correcto de APIs, mitigando vulnerabilidades comunes durante la fase de desarrollo.

SEMANA	TEMARIO	ACTIVIDADES DE APRENDIZAJE	EVIDENCIAS DE APRENDIZAJE	BIBLIOGRAFÍA
9 - 11	Seguridad en APIs I:- ¿Qué es una API y por qué es crítica para el negocio?	Sesión 9: Teoría: - APIs como contrato entre sistemas - Exposición de datos y		Bibliografía básica: OWASP. <i>OWASP API Security Top 10</i>. OWASP. <i>Authorization</i>



	Superficie de ataque en APIs

- Principales riesgos de seguridad en APIs
OWASP API Top 10 - Seguridad en APIs II:- Autenticación vs autorización en APIs

- Control de acceso a nivel de objeto y de operación

- Exposición excesiva de datos

- Casos reales de incidentes en APIs

- Principios de desarrollo seguro aplicados a APIs - Programación robusta y encapsulación:- Manejo seguro de errores y estados inválidos
- Reducción del impacto de fallas mediante diseño e implementación	funciones Laboratorio: - Ejercicios prácticos de OWASP TOP 10-I Sesión 10: Teoría - Tokens y contexto de seguridad (visión conceptual) - BOLA (Broken Object Level Authorization) -Validación del lado servidor Laboratorio - Ejercicios prácticos de OWASP TOP 10-II Sesión 11: Teoría - Programación defensiva y aislamiento de módulos Laboratorio	<i>Cheat Sheet.</i> OWASP. <i>REST Security Cheat Sheet.</i> McGraw, G. <i>Software Security: Building Security In.</i> Addison-Wesley. Martin, R. C. <i>Clean Architecture.</i> Prentice Hall. Bibliografía complementaria: Kohnfelder, L. <i>Designing Secure Software</i> .
--	---	---	--



Unidad 4: Análisis, pruebas y DevSecOps & Student Outcome 5

COMPETENCIA (S):

ABET 5 - Trabajo Multidisciplinario -

LOGRO DE LA UNIDAD: Al finalizar la unidad, el estudiante evaluará la seguridad del software mediante técnicas de análisis y pruebas, e integrará controles de seguridad en procesos automatizados de desarrollo y despliegue.

SEMANA	TEMARIO	ACTIVIDADES DE APRENDIZAJE	EVIDENCIAS DE APRENDIZAJE	BIBLIOGRAFÍA
12 - 16	- Análisis Estático y Dinámico:- Verificación de seguridad en el SDLC
- Análisis estático (SAST)
- Análisis dinámico (DAST)
- Complementariedad entre SAST y DAST
- Limitaciones y falsos positivos - Testing DevSecOps y supply chain:- Testing en el pipeline de desarrollo
- Unit testing, integration testing y security testing
- DevSecOps: integración de seguridad en CI/CD
-	SESIÓN 12: Teoría - Diferencias conceptuales entre SAST y DAST - ¿Qué detecta cada enfoque y qué no? - Análisis temprano vs análisis en ejecución - Seguridad como actividad continua - Relación entre requisitos, código y pruebas Laboratorio - Ejecución de análisis de seguridad SESIÓN 13: Teoría Práctica Calificada Laboratorio	• Evaluación de Desempeño 1 • Evaluación Final 1 • Práctica Calificada 2 • Trabajo 2	OWASP. <i>Web Security Testing Guide</i>. OWASP. <i>Application Security Verification Standard</i>. OWASP. <i>Top 10 CI/CD Security Risks</i>. OWASP. <i>Security Testing Report Guidelines</i> SLSA. <i>Supply-chain Levels for Software Artifacts</i>.



	Riesgos de la cadena de suministro de software
- Gestión de dependencias y librerías • Documentación y comunicación de seguridad:- Importancia de la documentación de seguridad
- Tipos de documentación de seguridad
- Comunicación técnica y ejecutiva
- Trazabilidad de riesgos y controles
- Presentación de resultados de seguridad	- Diseño de pruebas de seguridad SESIÓN 14: Teoria - Shift-left security - Testing como control preventivo - Riesgos de dependencias de terceros - Vulnerabilidades en librerías - Seguridad del pipeline como activo crítico Laboratorio - Análisis de pipeline y librerías SESIÓN 15: Teoria - Documentación como control de seguridad - Documentación técnica vs ejecutiva - Riesgos de mala comunicación - Evidencia de seguridad y auditoría		
--	--	---	--	--



		- Seguridad como insumo para toma de decisiones Laboratorio Presentación final de proyectos SESIÓN 16:		
--	--	---	--	--



VII. METODOLOGÍA

El Modelo Educativo de la UPC asegura una formación integral, la cual tiene como pilar el desarrollo de competencias. Estas son promovidas a través de un proceso de enseñanza-aprendizaje donde el estudiante cumple un rol activo en su aprendizaje. En este proceso dinámico, las competencias son construidas a partir de la reflexión crítica, el análisis, la discusión, la evaluación, la exposición y la interacción con sus pares, conectándolas con sus experiencias y conocimientos previos. Por ello, cada sesión está diseñada para ofrecer al estudiante diversas maneras de apropiarse y poner en práctica el nuevo conocimiento en contextos reales o simulados, reconociendo la importancia que esto tiene para su éxito profesional.

El curso está diseñado con una metodología de enseñanza que articula experiencias de aprendizaje sincrónicas (presenciales o virtuales) y actividades asincrónicas. En ambos casos, se aplica la dinámica de aula invertida. Es decir, la semana de aprendizaje comienza con actividades previas a la sesión presencial o virtual. Estas se enfocan en la lectura de uno o dos textos que constituyen la base del trabajo semanal y una actividad orientada a la verificación de dicho proceso de lectura. Luego de este proceso, se desarrolla la sesión del curso (que puede ser virtual o presencial, según la modalidad elegida en la matrícula), concentrada, por su parte, en la discusión de las lecturas, ya sean en plenario o en grupos de trabajo, a partir de casos específicos vinculados al área de estudio, que son propuestos por cada docente en particular.

VIII. EVALUACIÓN

-

Fórmula

$$: 0.1 * PC1 + 0.1 * TB1 + 0.2 * EA1 + 0.1 * PC2 + 0.15 * TB2 + 0.15 * DD1 + 0.2 * EB1$$

Cronograma

TIPO	COMPETENCIA	PESO	SEMANA	OBSERVACIÓN	RECUPERABLE
Práctica Calificada 1-PC1		10%	4		X
TIPO	COMPETENCIA	PESO	SEMANA	OBSERVACIÓN	RECUPERABLE
Trabajo 1-TB1		10%	7		
TIPO	COMPETENCIA	PESO	SEMANA	OBSERVACIÓN	RECUPERABLE



Evaluación Parcial 1-EA1		20%	8		
TIPO	COMPETENCIA	PESO	SEMANA	OBSERVACIÓN	RECUPERABLE
Práctica Calificada 2- PC2		10%	12		X
TIPO	COMPETENCIA	PESO	SEMANA	OBSERVACIÓN	RECUPERABLE
Trabajo 2-TB2		15%	15		
TIPO	COMPETENCIA	PESO	SEMANA	OBSERVACIÓN	RECUPERABLE
Evaluación de Desempeño 1- DD1		15%	15		
TIPO	COMPETENCIA	PESO	SEMANA	OBSERVACIÓN	RECUPERABLE
Evaluación Final 1-EB1		20%	16		



IX. BIBLIOGRAFÍA DEL CURSO

Para acceder a la bibliografía del curso, copia y pega el siguiente enlace en una nueva ventana del navegador:

- https://upc.alma.exlibrisgroup.com/leganto/readinglist/lists/12692061570003391?institute=51UPC_INST&auth=LOCAL

X. RECURSOS TECNOLÓGICOS

Generales

Para un óptimo desempeño en las sesiones de clases, y de acuerdo con el Reglamento de Estudios, el estudiante necesitará una computadora con un procesador de al menos 4.1 GHz, 8 GB de RAM, y un sistema operativo Windows 10 o macOS 10 en adelante. Es indispensable contar con una conexión a internet estable de 15 MB/s de descarga y 5MB/s de subida. Además, necesitará un navegador web compatible (Chrome, Edge, Firefox o Safari), un headset con micrófono y una cámara web. Asegurar el tener acceso al LMS de la universidad para acceder a los materiales y actividades de los cursos.

Esta configuración le permitirá participar activamente en las clases, acceder a todos los recursos y tener una experiencia de aprendizaje en línea o presencial fluida y efectiva.

Específicos del curso

XI. Anexos

En este anexo, encontrará los reglamentos que todo alumno está obligado a leer y a cumplir en su rol de estudiante universitario en UPC.

REGLAMENTO DE DISCIPLINA DE LOS ALUMNOS: <https://sica.upc.edu.pe/categoria/reglamentos-upc/sica-reg-26-reglamento-de-disciplina-de-alumnos>

REGLAMENTO PARA LA PREVENCIÓN E INTERVENCIÓN EN CASOS DE HOSTIGAMIENTO SEXUAL - UPC: <https://sica.upc.edu.pe/categoria/normalizacion/sica-reg-31-reglamento-para-la-prevencion-e-intervencion-en-casos-de-hostiga>